

Data Validation Dim 5 to 8

Outlier Detection, Timeliness, Distribution Profile, Relationships



Data Quality Metrics — Quantification Summary

From the lecture slides — here is the complete scoring table for all 8 dimensions:

Dimension	What It Measures	Formula
Accuracy	How closely values match ground truth	$(\text{correct values} / \text{total}) \times 100\%$
Consistency	Uniformity across systems	$(\text{conforming records} / \text{total}) \times 100\%$
Completeness	Required data present	$(\text{non-null values} / \text{total}) \times 100\%$
Uniqueness	Absence of duplicates	$(\text{unique records} / \text{total}) \times 100\%$
Outliers	Proportion of anomalous values	$(\text{non-outlier records} / \text{total}) \times 100\%$
Timeliness	Data freshness and latency	Mean time from event to availability
Distribution	Current vs expected distribution	KS statistic (0 = identical)
Relationships	Expected correlations between fields	Pearson or Spearman r

Dimension 5 — Outlier Detection

What is an Outlier?

An **outlier** is a data point that differs significantly from other observations. In data quality, outliers are important because they can:

- 🚨 Indicate **data entry errors** (e.g., age = 999)
- 🚨 Indicate **measurement errors** (e.g., sensor malfunction)
- ✅ Be **genuine rare events** worth keeping (e.g., a fraudulent transaction)

"If customer purchase amounts typically range from \$10–\$100, a transaction of \$50,000 warrants investigation."

Three Key Methods Covered:

Method	Best For	Assumption
Z-Score	Normally distributed data	Data is roughly bell-shaped
IQR (Interquartile Range)	Skewed / univariate data	No distributional assumption
Isolation Forest	Multi-dimensional, complex datasets	No assumption



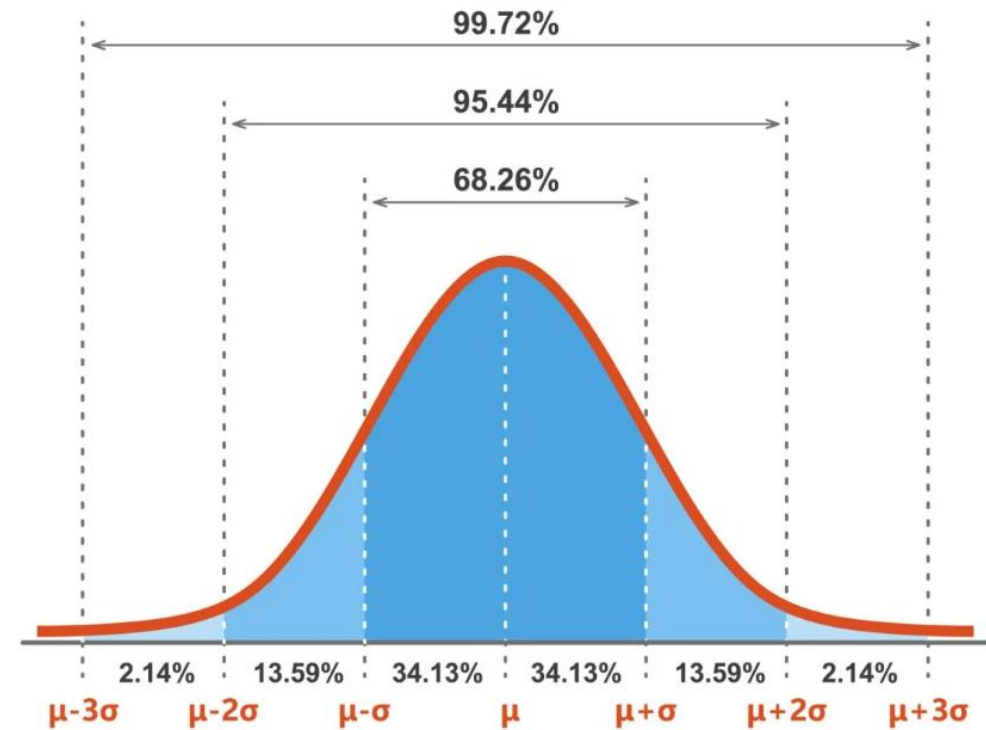
Method 1: Z-Score

$$z = \frac{x - \mu}{\sigma} \quad \text{Where:}$$

- x = individual value
- μ = mean of the dataset
- σ = standard deviation

Interpretation

Z-Score Range	Meaning
$-2 \leq z \leq 2$	Normal (covers ~95.4% of data)
$-3 \leq z \leq 3$	Acceptable edge (covers ~99.7% of data)
$ z > 3$	Likely outlier ← typical threshold
$ z > 2$	Suspicious ← stricter threshold



Range of z-score: $(-\infty, +\infty)$ — but practically, most normal data falls within ± 3 .

EXAMPLE 1 — Employee Salaries (mostly normal, one extreme error)

```
from scipy.stats import zscore

salaries_df['z_score'] = zscore(salaries_df['salary_USD'], ddof=1)
salaries_df['is_outlier_z3'] = np.abs(salaries_df['z_score']) > 3
salaries_df['is_outlier_z2'] = np.abs(salaries_df['z_score']) > 2
```

 Full DataFrame with Z-Scores:

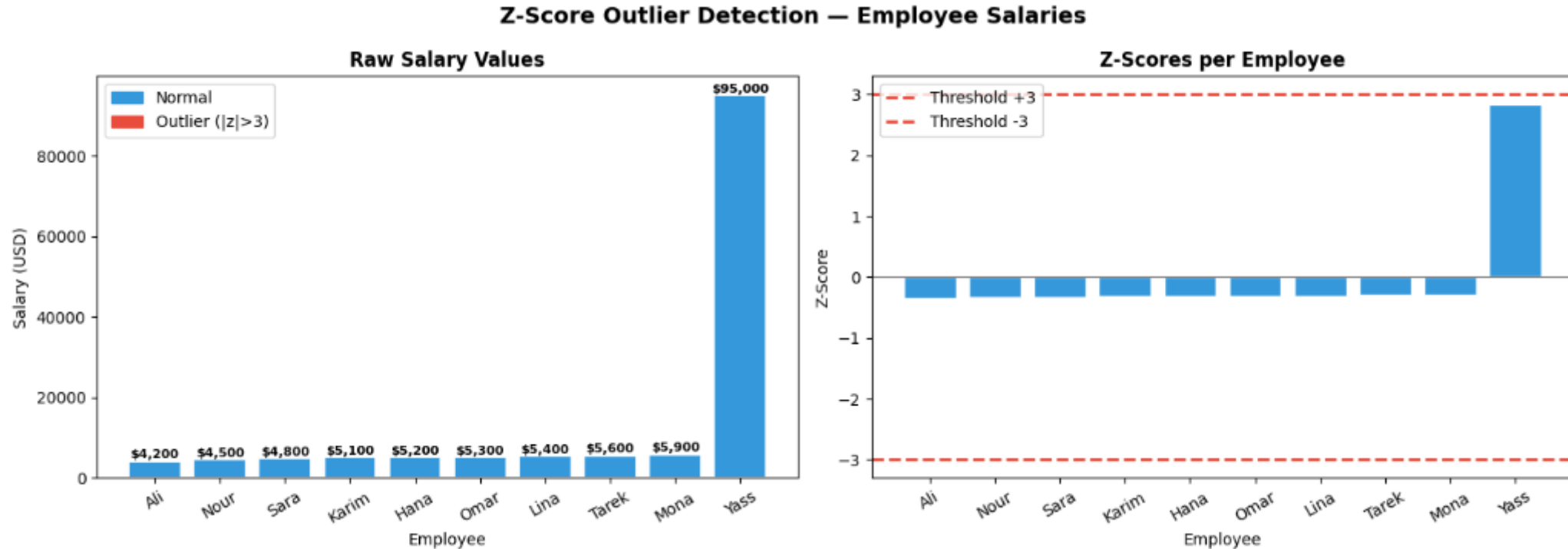
employee_id	name	salary_USD	z_score	is_outlier_z3	is_outlier_z2
E01	Ali	4200	-0.348226	False	False
E02	Nour	4500	-0.337673	False	False
E03	Sara	4800	-0.327121	False	False
E04	Karim	5100	-0.316569	False	False
E05	Hana	5200	-0.313051	False	False
E06	Omar	5300	-0.309534	False	False
E07	Lina	5400	-0.306017	False	False
E08	Tarek	5600	-0.298982	False	False
E09	Mona	5900	-0.288429	False	False
E10	Yass	95000	2.845602	False	True

 Mean salary : \$14,100

 Std deviation: \$28,430

Are the mean and std logical ?

EXAMPLE 1 — Employee Salaries (mostly normal, one extreme error)



Mean salary : \$14,100

Std deviation: \$28,430



Method 2: IQR — Interquartile Range

The IQR method is **non-parametric** (makes no assumption about the distribution). It uses the **spread of the middle 50%** of data to define what's "normal".

How It Works

Q_1 = 25th percentile (bottom quarter)

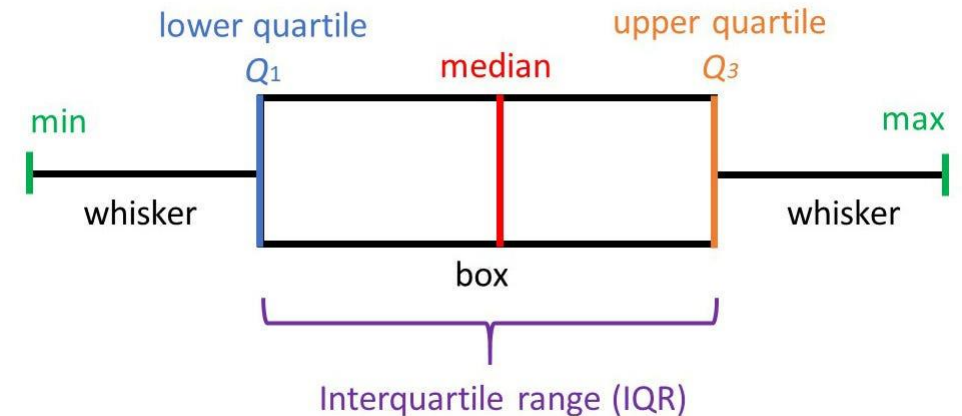
Q_3 = 75th percentile (top quarter)

$IQR = Q_3 - Q_1$ (middle 50% span)

Lower bound = $Q_1 - 1.5 \times IQR$

Upper bound = $Q_3 + 1.5 \times IQR$

Any value BELOW lower_bound or ABOVE upper_bound → Outlier



Percentiles vs. Quartiles vs. Quantiles

Same concept, different **granularity**:

Term	Divides data into	Unit
Percentiles	100 equal parts	P1, P2, ... P99
Quartiles	4 equal parts	Q1, Q2, Q3
Quantiles	any number of parts	0.0 → 1.0 (general term)

Quantile is the **umbrella term** — quartiles and percentiles are just specific types of quantiles:

- Quartile 1 = Quantile 0.25 = Percentile 25
- Quartile 2 = Quantile 0.50 = Percentile 50
- Quartile 3 = Quantile 0.75 = Percentile 75

How to Calculate Quartiles

Quartiles split sorted data into 4 equal parts.

Sort the data first:

[2, 4, 6, 8, 10, 12, 14, 16]

$n = 8$

Median (Q2) first — split the data in half:

Lower half: [2, 4, 6, 8]

Upper half: [10, 12, 14, 16]

$Q2 = (8 + 10) / 2 = 9$

Q1 = median of the lower half:

[2, 4, 6, 8]

$Q1 = (4 + 6) / 2 = 5$

Q3 = median of the upper half:

[10, 12, 14, 16]

$Q3 = (12 + 14) / 2 = 13$

IQR = Q3 - Q1:

$IQR = 13 - 5 = 8$

The rule: for n values, the median splits at position $(n+1)/2$. If that lands between two values, average them — which is why even-sized datasets produce decimal quartiles.

EXAMPLE 1 — House Prices in EGP (skewed, with luxury outliers)

```
Q1 = houses_df['price_EGP'].quantile(0.25)
Q3 = houses_df['price_EGP'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
```

```
houses_df['outlier_IQR'] = (houses_df['price_EGP'] < lower) | (
    houses_df['price_EGP'] > upper
)
```

```
print(f"\n🚨 Outliers detected: {houses_df['outlier_IQR'].sum()}")
```

EXAMPLE 1 — House Prices in EGP (skewed, with luxury outliers)

 House Price Table:

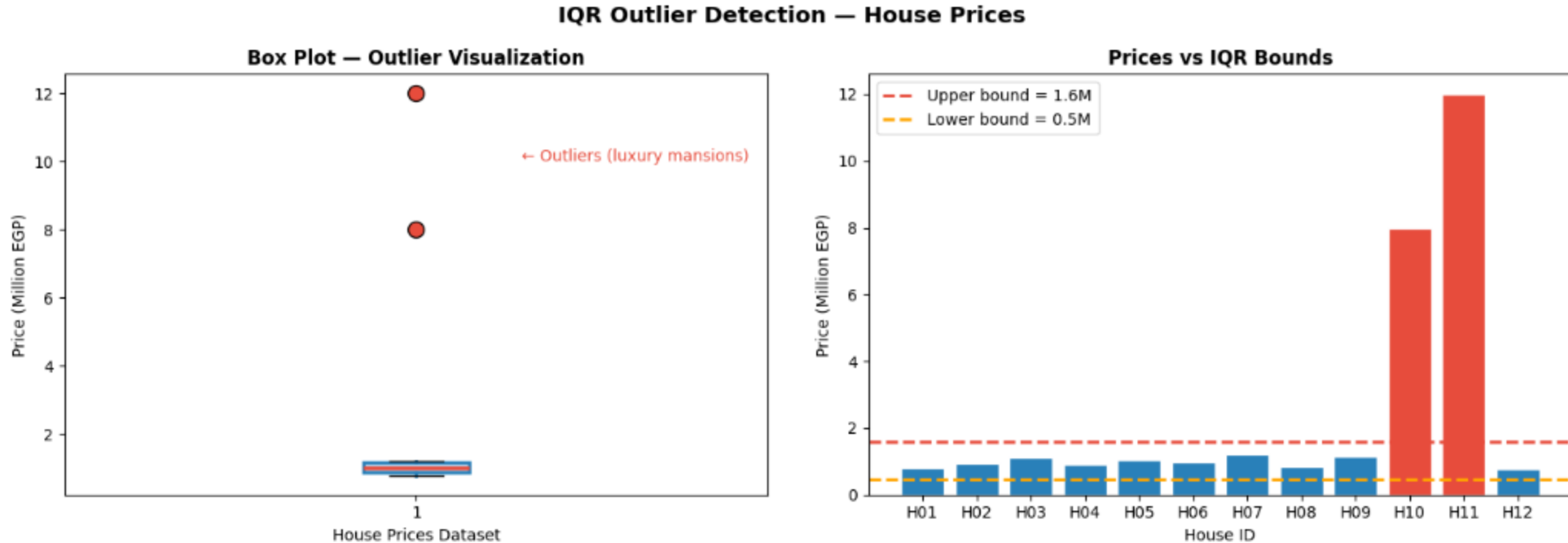
house_id	area_m2	price_EGP	outlier_IQR
H01	80	800000	False
H02	95	950000	False
H03	110	1100000	False
H04	90	900000	False
H05	105	1050000	False
H06	100	980000	False
H07	120	1200000	False
H08	85	850000	False
H09	115	1150000	False
H10	300	8000000	True
H11	350	12000000	True
H12	75	780000	False

 Q1 : EGP 887,500
 Q3 : EGP 1,162,500
 IQR : EGP 275,000
 Lower bound : EGP 475,000
 Upper bound : EGP 1,575,000

 Outliers detected: 2

house_id	area_m2	price_EGP
H10	300	8000000
H11	350	12000000

EXAMPLE 1 — House Prices in EGP (skewed, with luxury outliers)



💡 **KEY INSIGHT:** IQR works well here because the distribution is RIGHT-SKEWED. Z-score would be less reliable due to the non-normal shape.



Method 3: Isolation Forest

Core Idea

Isolation Forest is a **machine learning** approach for anomaly detection. Instead of building a profile of "normal" and then detecting deviations, it **directly isolates anomalies**.

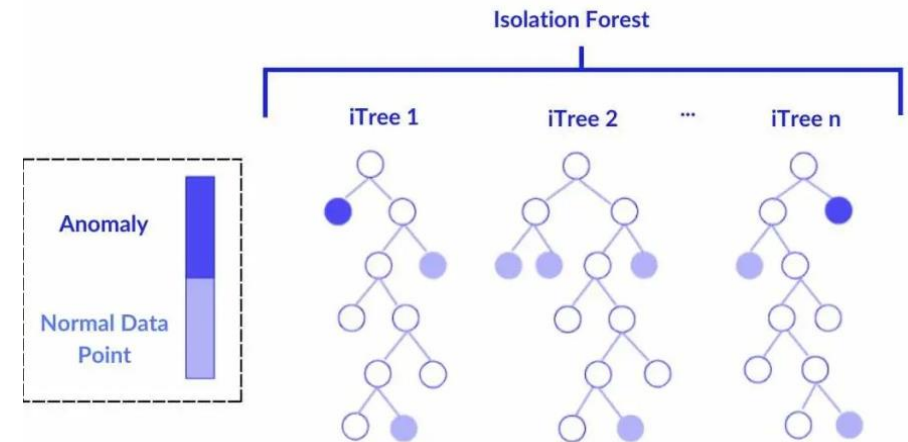
How It Works (Step-by-Step)

1. **Pick a random feature** from the dataset.
2. **Pick a random split value** within that feature's min-max range.
3. **Recursively split** until each data point is isolated.
4. **Count the path length** (number of splits) to isolate each point.

🔑 **Key Insight:** Anomalies are *rare* and *different*, so they require **fewer splits** to isolate → shorter path lengths → lower anomaly score.

Normal point: Requires 10-15 splits to isolate (deep in the tree)

Anomaly: Requires 2-3 splits to isolate (isolated quickly near root)



Output Values

Output	Meaning
+1	Inlier — normal point
-1	Outlier — anomaly detected

EXAMPLE 1: 2D Transaction Data — Where Isolation Forest excels

(both amount AND hour need to be jointly unusual)

```
from sklearn.ensemble import IsolationForest
```

```
features = ['amount_USD', 'hour_of_day']

clf = IsolationForest(
    n_estimators=200,    # 200 trees for stable results
    contamination=0.1,  # we expect ~10% outliers
    random_state=42,
)
```

```
transactions_df['anomaly_score'] = clf.fit_predict(transactions_df[features])
transactions_df['is_outlier'] = transactions_df['anomaly_score'] == -1
```

How Isolation Forest Works with Multiple Features

Instead of analyzing each feature separately, it sees all features as **coordinates in space** — each row becomes a point.

📄 Transaction Table:

tx_id	amount_USD	hour_of_day	anomaly_score	is_outlier
TX01	120	10	1	False
TX02	85	14	1	False
TX03	200	11	1	False
TX04	145	9	1	False
TX05	95	15	1	False
TX06	175	12	1	False
TX07	110	10	1	False
TX08	90	16	-1	True
TX09	160	13	1	False
TX10	130	11	1	False
TX11	5000	3	-1	True
TX12	180	14	1	False
TX13	115	10	1	False

🚨 Anomalies detected: 2

tx_id	amount_USD	hour_of_day
TX08	90	16
TX11	5000	3

How Isolation Forest Works with Multiple Features

Instead of analyzing each feature separately, it sees all features as **coordinates in space** — each row becomes a point.

Outliers get isolated in fewer splits because they sit far from the dense cluster. Normal points need many more splits to separate them.

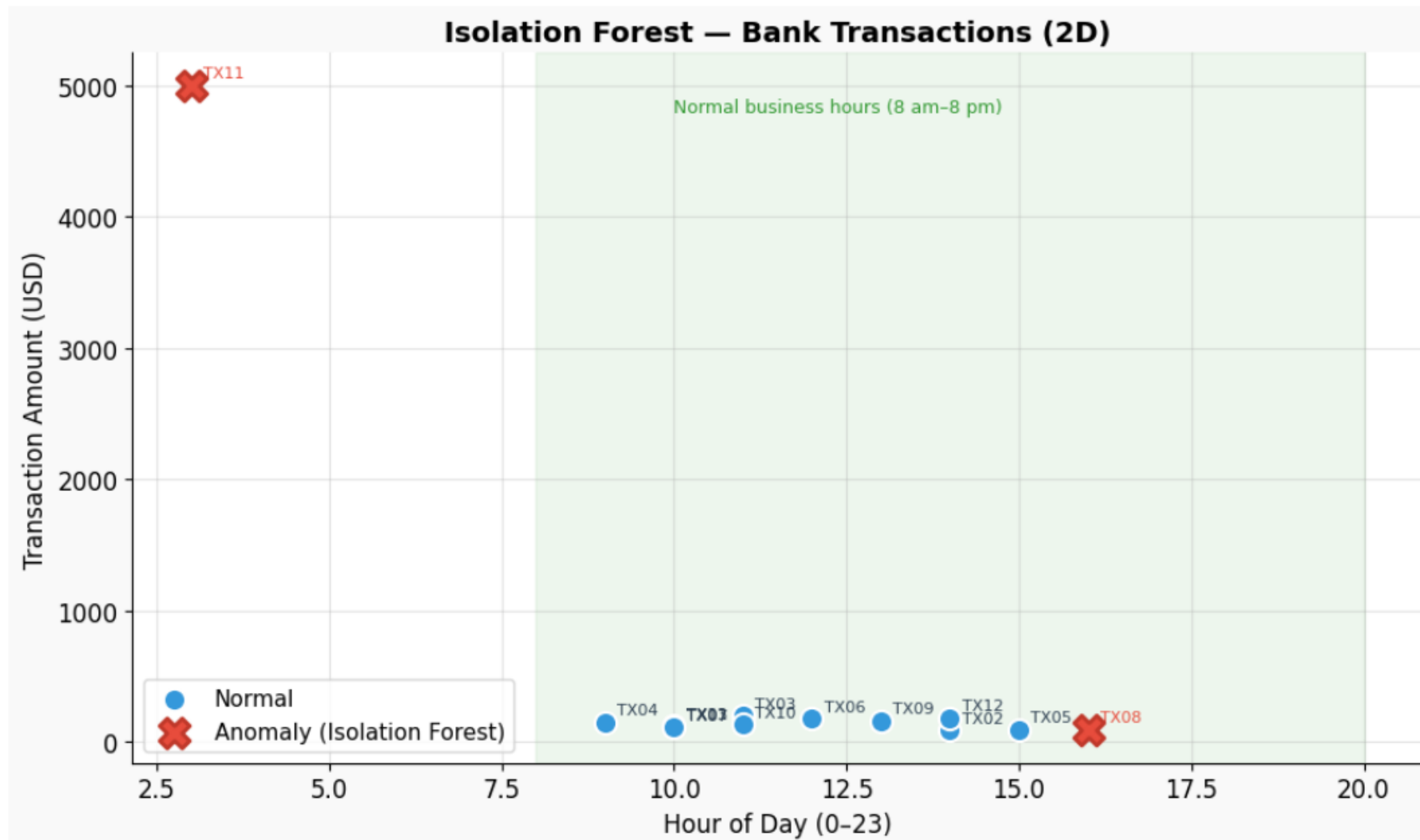
The joint detection power:

This is where multi-feature shines — consider these two cases:

TX-A: amount=5000, hour=14 → large amount, but business hours
→ maybe normal (big purchase)

TX-B: amount=5000, hour=3 → large amount AND 3am
→ jointly suspicious = isolated faster

A single-feature IQR on `amount` would flag both equally. Isolation Forest flags TX-B harder because **both dimensions are unusual together**.



💡 KEY INSIGHT: TX11 (\$5000 at 3 AM) is anomalous not just because of amount, but because of the COMBINATION of high amount + unusual hour. Z-Score or IQR on amount ALONE might miss this context!

=====

EXAMPLE 2 – Student Performance (3 features)

=====

📋 Student Table:

student	study_hrs	attendance	grade	anomaly	is_outlier
A	5	90	75	1	False
B	4	85	70	1	False
C	6	92	82	1	False
D	5	88	78	1	False
E	3	82	65	-1	True
F	7	95	88	1	False
G	4	87	72	1	False
H	5	89	77	1	False
I	6	91	80	1	False
J	0	20	95	-1	True
K	5	88	76	1	False
L	6	90	81	1	False

💣 Anomalies: 2

student	study_hrs	attendance	grade
E	3	82	65
J	0	20	95

💡 Student J: 0 study hours, 20% attendance, yet scored 95 – a suspicious pattern!

Comparing All Three Outlier Methods

Feature	Z-Score	IQR	Isolation Forest
Assumption	Normal distribution	None	None
Univariate / Multivariate	Univariate	Univariate	Multivariate
Skewed data	✗ Unreliable	✓ Good	✓ Good
Interpretability	✓ Very intuitive	✓ Intuitive	⚠ Less interpretable
Speed	✓ Fast	✓ Fast	⚠ Slower (ML model)
Detects complex anomalies	✗	✗	✓
Code complexity	Simple	Simple	Moderate

When to Use Each Method

-  **Z-Score:** Quick sanity check on approximately normal data (e.g., heights, IQ scores)
-  **IQR:** Skewed distributions (e.g., income, house prices, purchase amounts)
-  **Isolation Forest:** Multiple features where the *combination* creates the anomaly



Dimension 6 — Timeliness

What is Timeliness?

Timeliness validates **time-based properties** of your data. Data can be accurate, complete, and consistent — but if it's stale, it's useless (or harmful) for decision-making.

What to Validate

Check	What It Means	Example
Data freshness	Is the latest record recent enough?	Sensor data must arrive within last 5 minutes
Chronological order	Are timestamps sequential?	Events should not appear before they were triggered
Expected frequency	Does data arrive at expected rate?	API should produce 100 events/hour
Gap detection	Are there unexpected holes in time?	Missing day in daily sales records
Duplicate timestamps	Same moment recorded twice?	Two records with identical timestamp

Key pandas/datetime tools

Function	Purpose
<code>pd.to_datetime()</code>	Convert strings/integers to datetime
<code>pd.Timestamp.now()</code>	Get current datetime
<code>pd.Timedelta()</code>	Create time duration objects
<code>df['ts'].is_monotonic_increasing</code>	Check if timestamps are in order
<code>df['ts'].diff()</code>	Time gap between consecutive rows
<code>.dt.total_seconds()</code>	Convert timedelta to seconds
<code>pd.date_range()</code>	Generate expected time sequence

```
# — CHECK 1: Freshness —————
latest_ts = sensor_readings['timestamp'].max()
simulated_now = pd.Timestamp('2026-03-04 09:00:00') # simulating "now"
age_minutes = (simulated_now - latest_ts).total_seconds() / 60

print(f"\n✅ CHECK 1 – FRESHNESS")
print(f"    Latest record : {latest_ts}")
print(f"    Current time   : {simulated_now}")
print(f"    Data age       : {age_minutes:.0f} minutes")
if age_minutes > 30:
|     print("    ⚠️ WARNING: Data is more than 30 minutes old!")
else:
|     print("    ✅ Data is fresh (within 30-min threshold)")
```

```
# — CHECK 2: Chronological Order —————
print(f"\n✅ CHECK 2 – CHRONOLOGICAL ORDER")
is_ordered = sensor_readings['timestamp'].is_monotonic_increasing
print(f"    is_monotonic_increasing = {is_ordered}")
if not is_ordered:
    out_of_order = sensor_readings[
        sensor_readings['timestamp'] < sensor_readings['timestamp'].shift(1)
    ]
    # shift(1) gives you the row above (previous) in the same column.
    # so the whole column data shift(1) means shifted one downward, stating previous
    # col:          10:00 12:00 13:00
    # shift(1):  NaT   10:00 12:00
    print(f"    ⚠️ WARNING: {len(out_of_order)} out-of-order record(s) found!")
    print(out_of_order[['reading_id', 'timestamp']].to_string(index=False))
```

=====

EXAMPLE 1 – IoT Sensor Readings: Freshness + Order Checks

=====

Raw Sensor Data:

reading_id	timestamp	temperature_C
1	2026-03-04 08:00:00	22.1
2	2026-03-04 08:05:00	22.3
3	2026-03-04 08:10:00	22.5
4	2026-03-04 08:03:00	22.2
5	2026-03-04 08:20:00	22.8
6	2026-03-04 08:25:00	23.0
7	2026-03-04 08:30:00	22.9
8	2026-03-04 08:35:00	23.1

CHECK 1 – FRESHNESS

Latest record : 2026-03-04 08:35:00

Current time : 2026-03-04 09:00:00

Data age : 25 minutes

 Data is fresh (within 30-min threshold)

CHECK 2 – CHRONOLOGICAL ORDER

is_monotonic_increasing = False

 WARNING: 1 out-of-order record(s) found!

reading_id	timestamp
4	2026-03-04 08:03:00

```

# — CHECK 3: Gap Detection —————
print("\n✅ CHECK 3 – GAP DETECTION")

sorted_readings = sensor_readings.sort_values('timestamp').reset_index(drop=True)
# This resets index back to 0, 1, 2, 3... cleanly.
# drop=True discards the old index instead of saving it as a column.

sorted_readings['time_gap_min'] = (
    sorted_readings['timestamp'].diff().dt.total_seconds() / 60
)
# .diff() – subtracts each row's value from the row below's value
# (i.e. current - previous), giving a Timedelta per row.
# col:      10:00 10:05 10:25
# diff():   NaT 0:05:00 0:20:00
# NaT = Not a Time – pandas' equivalent of NaN but for datetime/timedelta columns.

expected_gap_min = 5 # we expect a reading every 5 minutes
sorted_readings['gap_ok'] = (
    sorted_readings['time_gap_min'] <= expected_gap_min * 1.5
) | sorted_readings['time_gap_min'].isna()
# | – OR operator. A row is gap_ok = True if either condition is met
# – gap is within threshold, or it's the first row (no gap to measure).

```


✓ CHECK 3 – GAP DETECTION

📋 Sorted Readings with Time Gaps:

reading_id	timestamp	temperature_C	time_gap_min	gap_ok
1	2026-03-04 08:00:00	22.1	NaN	True
4	2026-03-04 08:03:00	22.2	3.0	True
2	2026-03-04 08:05:00	22.3	2.0	True
3	2026-03-04 08:10:00	22.5	5.0	True
5	2026-03-04 08:20:00	22.8	10.0	False
6	2026-03-04 08:25:00	23.0	5.0	True
7	2026-03-04 08:30:00	22.9	5.0	True
8	2026-03-04 08:35:00	23.1	5.0	True

⚠ WARNING: 1 unexpected large gap(s) detected!

— Find missing days —————

```
full_range = pd.date_range(  
    start=sales_df['date'].min(), end=sales_df['date'].max(), freq='D'  
)
```

generate sequence from start to end date, gap is 1D (1 day)

```
missing_days = full_range[~full_range.isin(sales_df['date'])]
```

.isin(sales_df['date']) – returns a boolean array:

True where a date from full_range exists in the sales data, False where it doesn't.

— Find duplicate dates —————

```
dup_dates = sales_df[sales_df.duplicated(subset='date', keep=False)]
```

scans the date column and returns a boolean array: True for every row
that shares a date with at least one other row.

```
# keep=False – marks "all" duplicate occurrences as True
```

(vs 'first'/'last' which would spare one copy).

so all duplicated values including the first one, are marked True

Default is keep='first' – marks all duplicates as True except the first occurrence.

=====

EXAMPLE 2 – Daily Sales: Missing Days + Duplicate Timestamps

=====

 Sales DataFrame:

date	sales_EGP
2026-02-01	12500
2026-02-02	13200
2026-02-03	11800
2026-02-05	14300
2026-02-06	9800
2026-02-06	9800
2026-02-07	15100
2026-02-08	13600

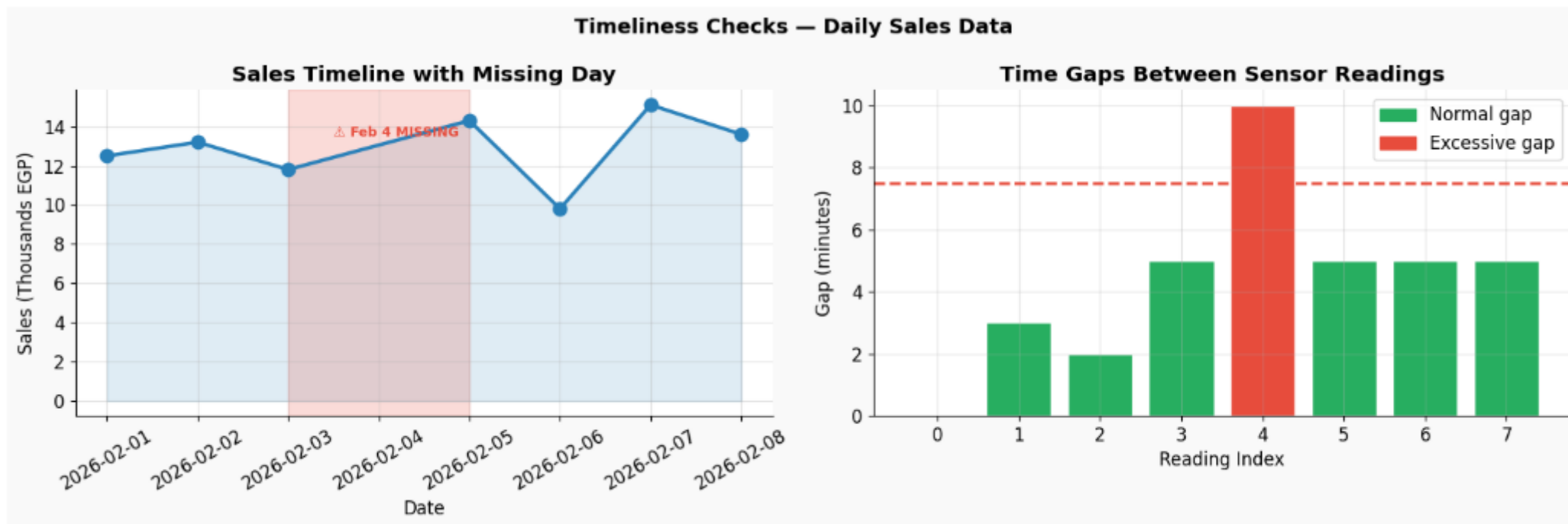
 Expected days : 8

 Recorded days : 7

 Missing days : ['2026-02-04']

 Duplicate date entries:

date	sales_EGP
2026-02-06	9800
2026-02-06	9800



💡 **KEY INSIGHT:** Missing days in sales data could mean system downtime, data pipeline failure, or simply a weekend/holiday – each requires different handling! Always INVESTIGATE before imputing.



Dimension 7 — Distribution Profile

What is a Distribution Profile?

A **distribution profile** examines how values are **spread** within a column. It helps you understand:

- What's the typical value? (central tendency)
- How spread out are the values? (dispersion)
- Is the distribution symmetric or skewed?
- Are there unusually heavy tails?

Key Statistics

Statistic	Formula / Function	What it tells you
Mean	$\bar{x} = \frac{\sum x_i}{n}$	Arithmetic average (sensitive to outliers)
Median	Middle value when sorted	Robust center (not affected by outliers)
Std (σ)	$\sigma = \sqrt{\frac{\sum (x - \bar{x})^2}{n}}$	Average spread from the mean
Min / Max	—	Range of values
Skewness	<code>scipy.stats.skew()</code>	Symmetry of distribution
Kurtosis	<code>scipy.stats.kurtosis()</code>	Peakedness vs. normal distribution



Central Tendency — Mean, Median, Mode



Median

Median is **quantile(0.50)** — the following are all equivalent:

```
df.median()  
df['col'].quantile(0.50)  
df['col'].median()
```



Quantiles

General quantile calculation:

```
df.quantile(0.5)  
df.quantile([0, 0.25, 0.50, 0.75, 1.0])  
df['col'].quantile(0.5)  
df['col'].quantile([0, 0.25, 0.50, 0.75, 1.0])
```



Mean

Mean is the **arithmetic average** — sensitive to outliers:

```
df.mean()  
df['col'].mean()
```



Mode

Mode is the **most frequent value** — use `iloc[0]` in case of multiple modes:

```
df.mode().iloc[0]           # first mode of each column  
df['col'].mode()[0]         # first mode of a single column
```



Skewness — Measuring Asymmetry

$$\text{Skewness} = \frac{1}{n} \sum \left(\frac{x_i - \bar{x}}{\sigma} \right)^3$$

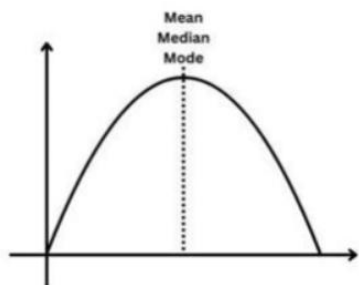
NOTE: Cubing is important because it preserves the sign.

Interpretation:

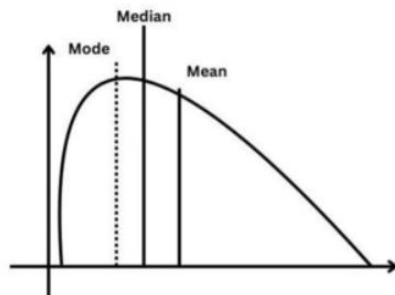
- 0 → symmetric
- > 0 → right-skewed (long tail to the right)
- < 0 → left-skewed (long tail to the left)

Skewness Value	Classification
< -1	Strongly negatively skewed
-1 to -0.5	Moderately negatively skewed
-0.5 to 0.5	Approximately symmetric ✓
0.5 to 1	Moderately positively skewed
> 1	Strongly positively skewed

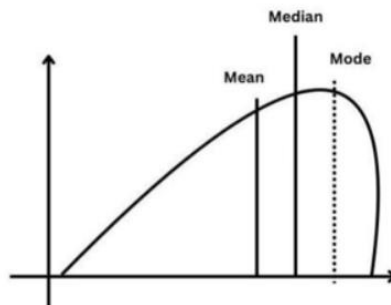
Symmetric (skew=0)



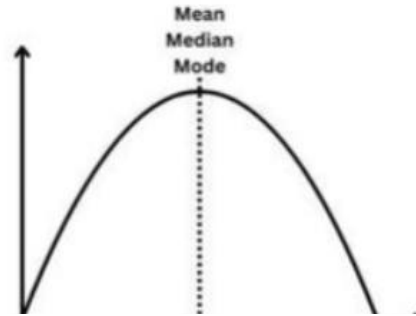
Positive Skew



Negative Skew



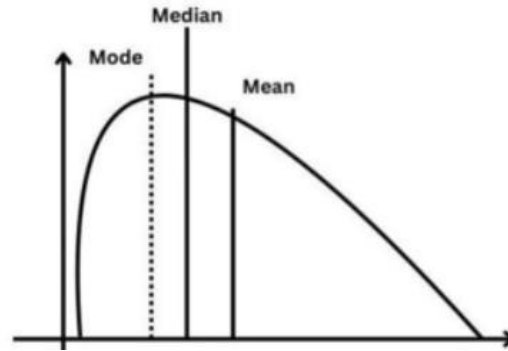
Symmetric (skew=0)



SYMMETRIC (skew ≈ 0)

Mean=Median=Mode

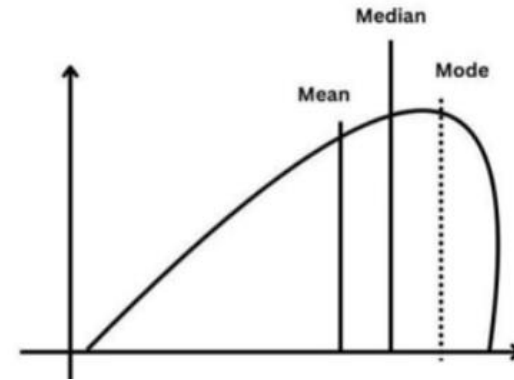
Positive Skew



POSITIVE SKEW (skew > 0)

Mode < Median < Mean
(tail $\rightarrow$ right)

Negative Skew



NEGATIVE SKEW (skew < 0)

Mean < Median < Mode
(tail $\rightarrow$ left)

Key intuition: the mean is the most sensitive to outliers — it always gets pulled toward the tail, dragging it away from the mode. The median resists this pull, sitting between them.

- **Mode** — the value that appears **most frequently**. It's the peak of the density curve — where most data is concentrated.
- **Mean** — the arithmetic average. It is the **balance point of the distribution**. It is **sensitive to outliers**. gets **pulled** toward the long **tail**. Even a few very large values (e.g. billionaires in income data) exert enormous leverage on the average. A single value of 1,000,000 can shift the mean dramatically.
- **Median** — the **middle value** when data is sorted. Exactly 50% of data falls below it, 50% above. It is **resistant to outliers**. It only cares about the **middle position** in sorted data — not how far extreme values are.

One sentence: the mean chases the tail because outliers move the balance point; the median ignores how far they are; the mode just stays at the peak.


```
# To make 1 df containing mean, median, mode:
```

```
stats_df = pd.DataFrame(  
    {  
        'mean': dist_df.mean(),  
        'median': dist_df.median(),  
        'mode': dist_df.mode().iloc[0],  
        'Q1': dist_df.quantile(0.25),  
        'Q2': dist_df.quantile(0.5),  
        'Q3': dist_df.quantile(0.75),  
    }  
)
```

```
dist_df.mean()    # → Series (one value per column)
```

```
dist_df.median()  # → Series (one value per column)
```

```
dist_df.mode()    # → DataFrame (could be multiple rows!)
```

```
#    col1  col2
```

```
# 0    25    30  ← first mode
```

```
# 1    40    NaN ← second mode (if tie exists)
```

```
...
```

A column can have **multiple modes** when two values appear equally often:

```
...
```

```
[1, 1, 2, 2, 3] → modes are both 1 and 2
```

```
print("\n📊 Skewness Values:")
```

```
for col in dist_df.columns:
```

```
    s = skew(dist_df[col])
```

```
    tag = (
```

```
        "≈ Symmetric"
```

```
        if abs(s) < 0.5
```

```
        else ("Positive Skew (right tail)" if s > 0 else "Negative Skew (left tail)")
```

```
    )
```

```
    print(f"    {col:20s}: skewness = {s:+.3f} → {tag}")
```

Skewness Demonstration – Three Designed Distributions

Descriptive Statistics:

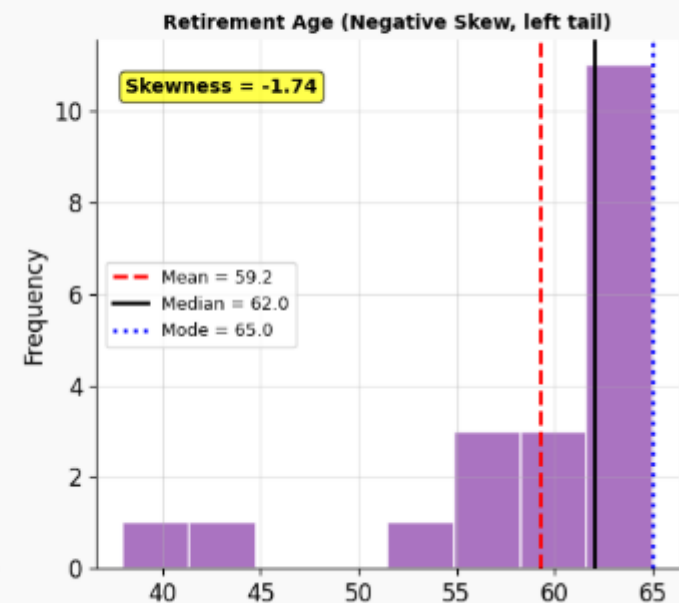
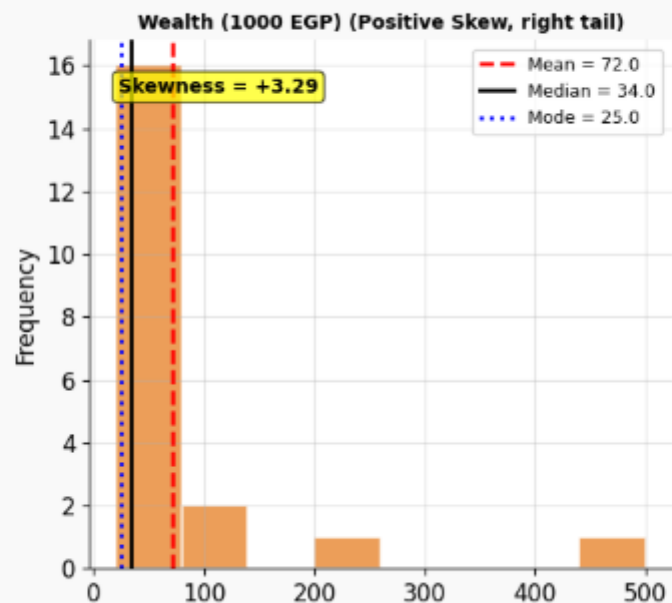
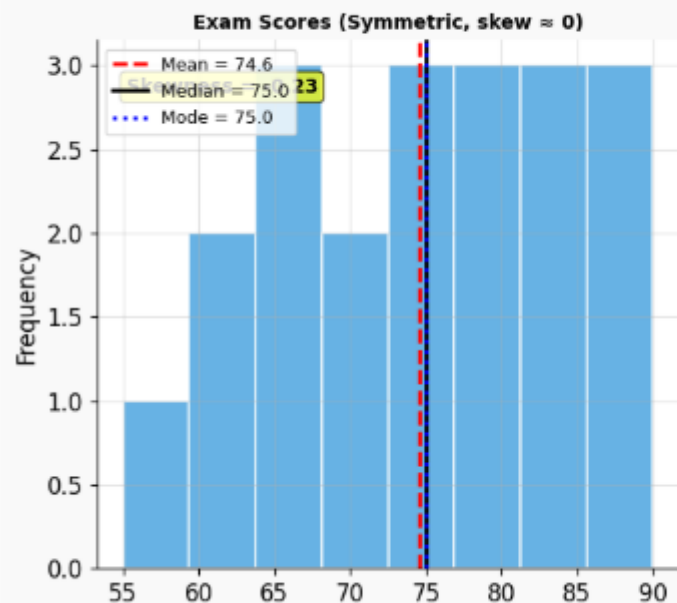
	exam_scores	wealth_kEGP	retirement_age
count	20.00	20.00	20.00
mean	74.60	72.05	59.25
std	9.82	109.30	7.55
min	55.00	20.00	38.00
25%	67.75	27.75	57.75
50%	75.00	34.00	62.00
75%	82.25	52.50	64.25
max	90.00	500.00	65.00
mode	75.00	25.00	65.00

	mean	median	mode	Q1	Q2	Q3
exam_scores	74.60	75.0	75	67.75	75.0	82.25
wealth_kEGP	72.05	34.0	25	27.75	34.0	52.50
retirement_age	59.25	62.0	65	57.75	62.0	64.25

Skewness Values:

exam_scores : skewness = -0.233 → ≈ Symmetric
wealth_kEGP : skewness = +3.288 → Positive Skew (right tail)
retirement_age : skewness = -1.740 → Negative Skew (left tail)

Skewness — Three Real-World Examples



KEY INSIGHT:

- When Mean $\approx$ Median $\approx$ Mode $\rightarrow$ roughly symmetric
- When Mean $>$ Median $>$ Mode $\rightarrow$ positive skew (long right tail)
- When Mean $<$ Median $<$ Mode $\rightarrow$ negative skew (long left tail)



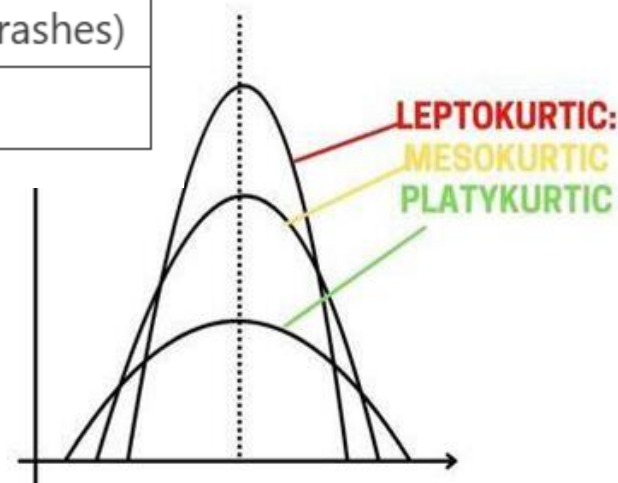
Kurtosis — Measuring Peakedness

$$\text{Kurtosis} = \frac{1}{n} \sum \left(\frac{x_i - \bar{x}}{\sigma} \right)^4 - 3$$

The **-3** term is the "excess kurtosis" correction, making the normal distribution have kurtosis = 0.

Three Types of Kurtosis

Type	Kurtosis Value	Shape	Example
Mesokurtic	≈ 0	Normal bell curve	IQ scores
Leptokurtic	> 0	Taller peak, heavier tails	Stock returns (fat tails = more crashes)
Platykurtic	< 0	Flatter, shorter peak	Uniform distribution



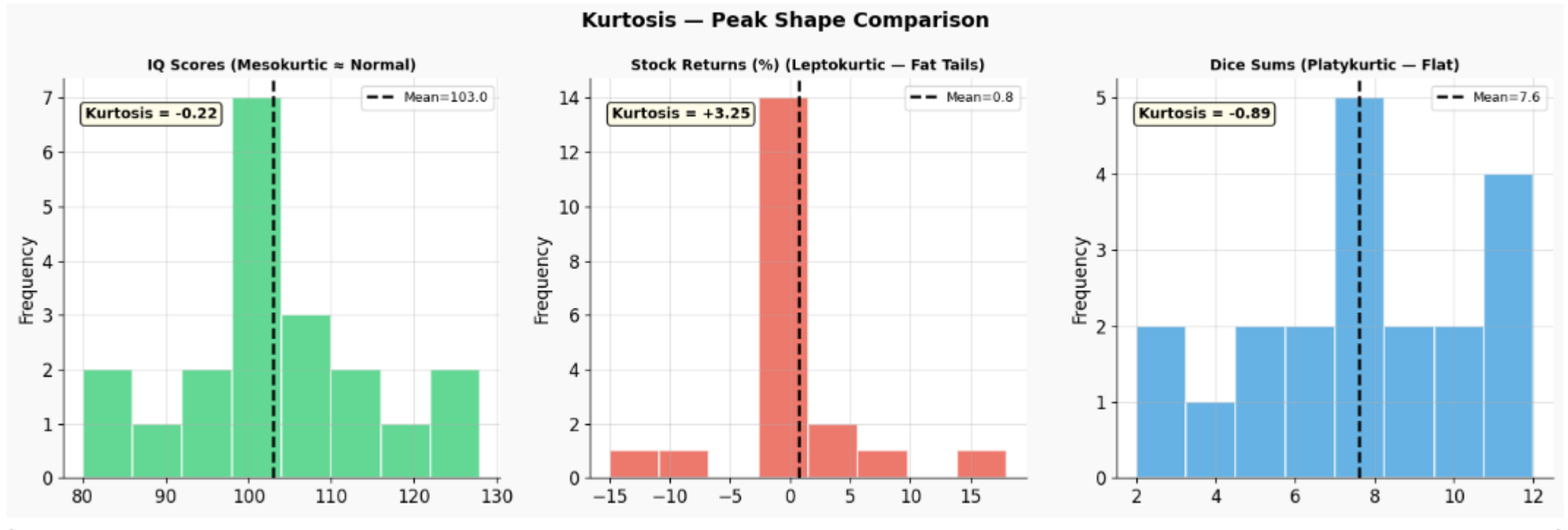
```

print("\n📊 Kurtosis Values (excess, normal=0):")
for col in kurt_df.columns:
    k = kurtosis(kurt_df[col])
    s = skew(kurt_df[col])
    tag = (
        "Mesokurtic ( $\approx$  normal)"
        if abs(k) < 0.5
        else ("Leptokurtic (heavy tails)" if k > 0 else "Platykurtic (light tails)")
    )
    print(f"    {col:22s}: kurtosis = {k:+.3f} → {tag} (skew={s:+.2f})")

```

📊 Kurtosis Values (excess, normal=0):

iq_scores	: kurtosis = -0.222	→ Mesokurtic ($\approx$ normal)	(skew=+0.23)
stock_returns_pct	: kurtosis = +3.249	→ Leptokurtic (heavy tails)	(skew=+0.28)
dice_sums	: kurtosis = -0.891	→ Platykurtic (light tails)	(skew=-0.18)



KEY INSIGHT for Data Validation:

- High positive kurtosis (leptokurtic) → MORE extreme values than expected
→ More likely to have severe outliers
- Negative kurtosis (platykurtic) → Values more spread evenly
→ Fewer extreme outliers but wider range

Kolmogorov-Smirnov (KS) Test — Distribution Comparison

The **KS test** checks if a dataset follows a specific distribution (one-sample) or if two datasets come from the same distribution (two-sample).

Use Case in Data Validation

 **Data Drift Detection:** Has your production data drifted away from the training distribution? If yes, your model may no longer be valid!

How It Works

The KS test compares the **Empirical Cumulative Distribution Function (ECDF)** of your data against a reference distribution.

ECDF: $F_n(x) = (\text{number of observations} \leq x) / n$

KS statistic: $D = \max |F_n(x) - F(x)|$

Kolmogorov-Smirnov (KS) Test — Distribution Comparison

KS Statistic	Meaning
≈ 0	Distributions are nearly identical ✓
$0.05-0.15$	Small difference
> 0.2	Significant difference ⚠
$\rightarrow 1$	Completely different distributions 🚨

```
scipy.stats.kstest()
```

Returns:

- `statistic` — KS distance (0 to 1)
- `pvalue` — If < 0.05 , reject H_0 (data does NOT follow the distribution)


```
ks_stat, p_val = kstest(
    exam_scores, 'norm', args=(np.mean(exam_scores), np.std(exam_scores))
)
```

=====

EXAMPLE 1 – Near-Normal Exam Scores → KS test should PASS

=====

Data : [58 62 65 67 69 70 71 72 73 74 75 75 76 77 78 79 80 82 83 85]
Mean : 73.55 | Std: 6.85

KS Statistic : 0.0838 (closer to 0 = more normal)
p-value : 0.9967

✅ RESULT: Not Reject H_0 → Data is consistent with Normal distribution

=====

EXAMPLE 2 – Corrupted Scores (extreme outliers) → KS test FAILS

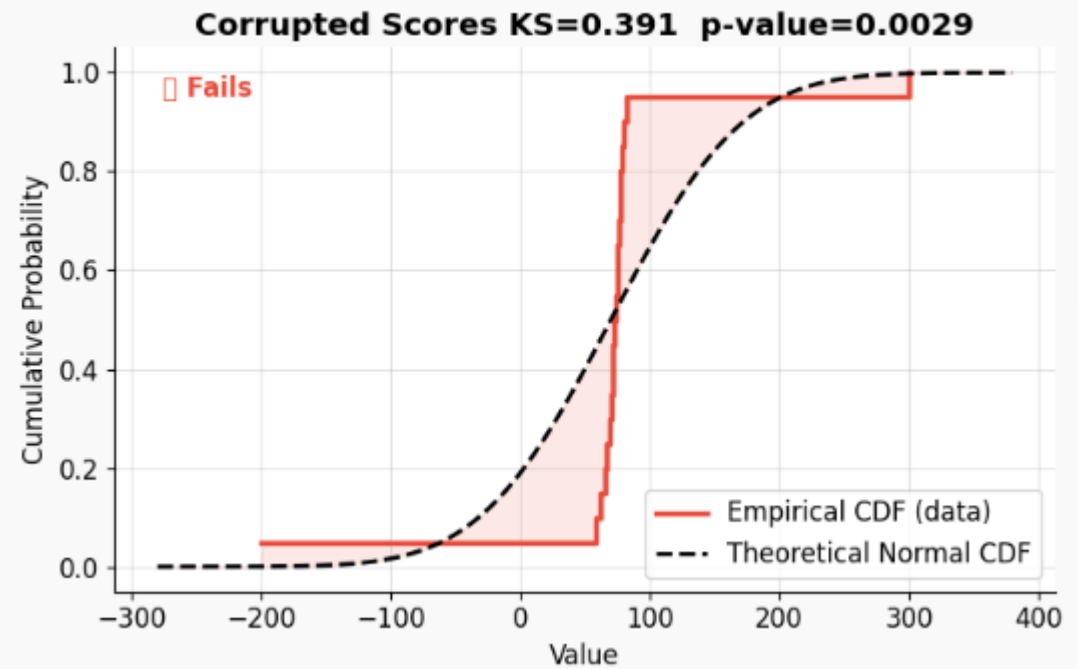
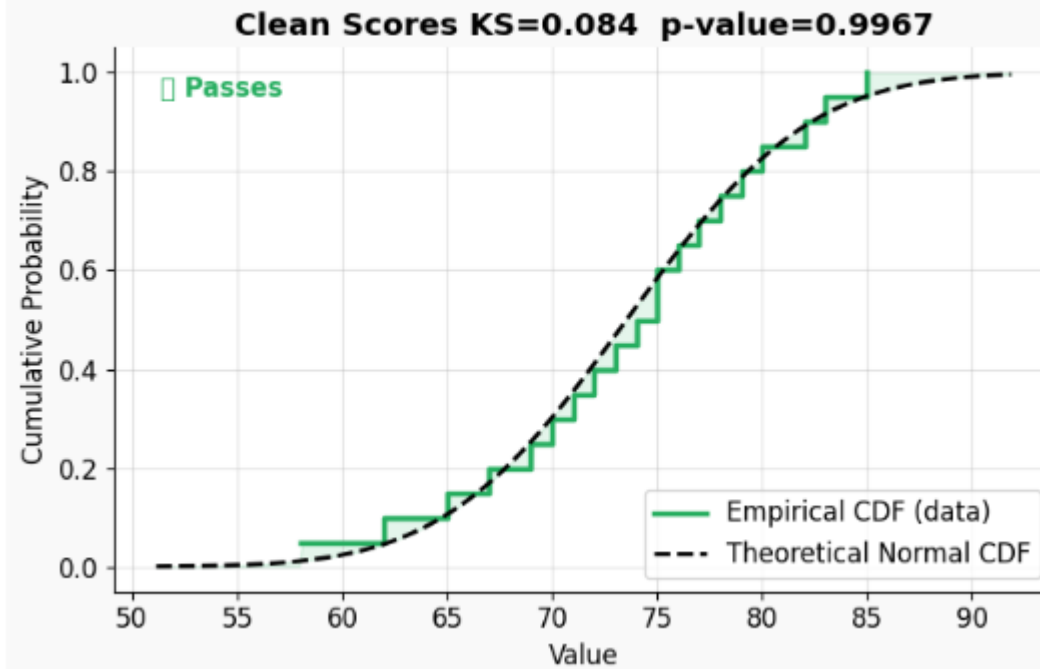
=====

Data : [58 62 65 67 69 70 71 72 73 74 75 75 76 77
78 79 80 82 -200 300]
Mean : 70.15 | Std: 79.56 ← pulled by outliers!

KS Statistic : 0.3908 (larger = more deviation)
p-value : 0.002910

🚨 RESULT: Reject H_0 → Data does NOT follow Normal distribution!
→ Investigate the extreme values (-200, 300) – likely data entry errors!

KS Test — ECDF vs Theoretical Normal CDF



- 💡 The KS DISTANCE is the maximum vertical gap between the two CDF curves.
Larger gaps → more evidence the data does NOT follow the expected distribution.



Dimension 8 — Relationships & Correlation

Why Validate Relationships?

Examining **correlations and dependencies** between fields helps uncover:

- 🚨 **Data quality issues:** Two fields that should be independent are correlated
- 🚨 **Multicollinearity:** Highly correlated features that destabilize models
- ✅ **Expected patterns:** Customer ID correlating with purchase amount → IDs not randomly assigned
- ✅ **Implied relationships:** ZIP code → City, State (affects normalization)

Two Main Methods

Method	Type	Assumption	Best For
Pearson	Linear	Data is normally distributed	Continuous numerical data
Spearman	Monotonic	No distributional assumption	Ordinal / non-normal / outliers present



Method 1: Pearson Correlation

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \cdot \sigma_Y} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

Interpretation

r Value	Strength	Direction
+1.0	Perfect	Positive (as X↑, Y↑)
+0.7 to +1.0	Strong Positive	—
+0.4 to +0.7	Moderate Positive	—
+0.1 to +0.4	Weak Positive	—
0	No linear relationship	—
-0.1 to -0.4	Weak Negative	—
-0.4 to -0.7	Moderate Negative	—
-0.7 to -1.0	Strong Negative	—
-1.0	Perfect	Negative (as X↑, Y↓)

Variance — how much a **single feature** spreads from its mean:

How volatile is price alone?

Covariance — how much **two features** move together:

When price goes up, does area go up too?

Covariance > 0 → both go up together

Covariance < 0 → one goes up, other goes down

Covariance = 0 → no relationship

Variance is just covariance of a **feature with itself**.

```
pearson_matrix = corr_df.corr(method='pearson').round(3)
print("\n📄 Pearson Correlation Matrix:")
print(pearson_matrix)
```

=====

EXAMPLE 1 – Pearson Correlation: 4 relationship types

=====

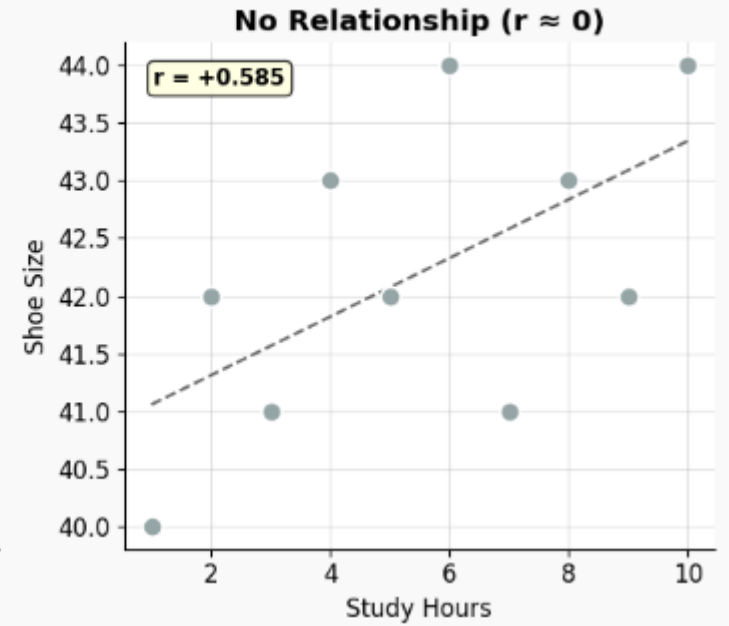
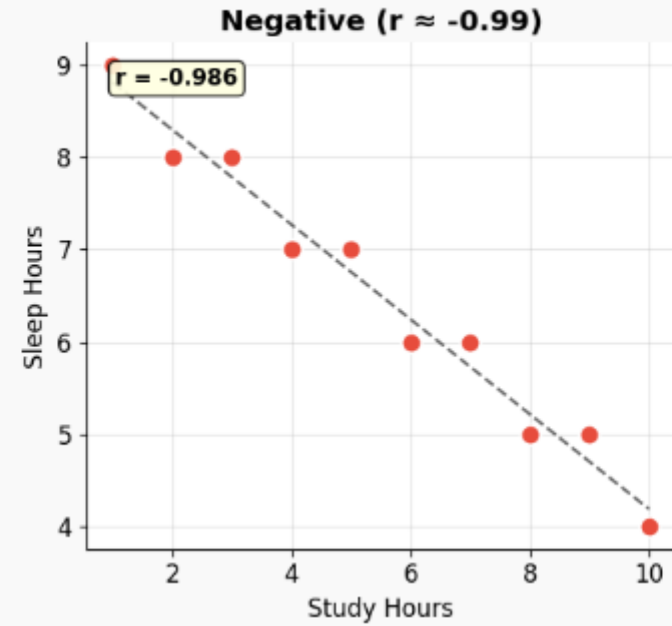
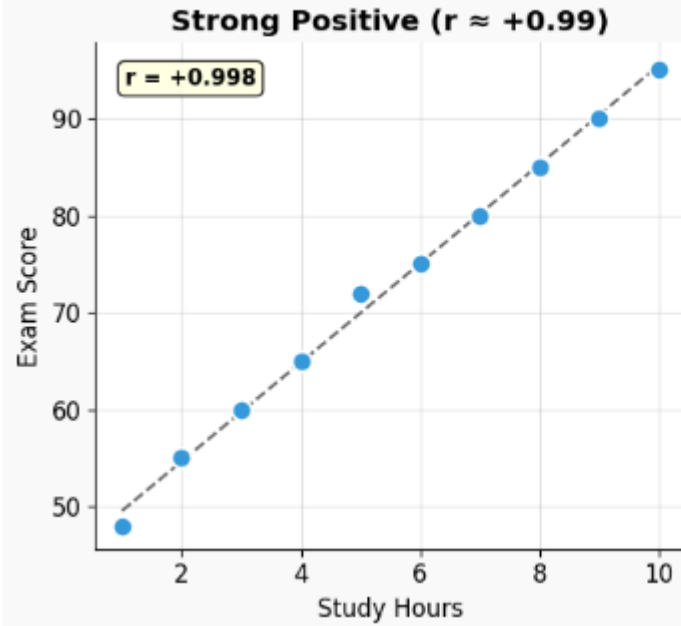
📄 Pearson Correlation Matrix:

	study_hours	exam_score	sleep_hours	shoe_size	hours_squared
study_hours	1.000	0.998	-0.986	0.585	0.975
exam_score	0.998	1.000	-0.984	0.595	0.965
sleep_hours	-0.986	-0.984	1.000	-0.694	-0.961
shoe_size	0.585	0.595	-0.694	1.000	0.531
hours_squared	0.975	0.965	-0.961	0.531	1.000

📊 Key Correlations with exam_score:

exam_score vs study_hours : $r = +0.998$
exam_score vs sleep_hours : $r = -0.984$
exam_score vs shoe_size : $r = +0.595$
exam_score vs hours_squared : $r = +0.965$

Pearson Correlation — Visual Comparison



KEY INSIGHT: Pearson ONLY captures LINEAR relationships.

A curved relationship may have $r \approx 0$ even if the two variables are strongly related!

Method 2: Spearman's Rank Correlation

Spearman converts values to ranks and then computes Pearson correlation on those ranks.

Student	Hours Studied	Exam Score	Rank (Hours)	Rank (Score)
A	8	90	4	3
B	2	60	1	1
C	10	93	5	5
D	4	85	2	2
E	6	92	3	4

Ranks are assigned by **sorting each column independently** from smallest to largest — the original row order doesn't matter. Notice the ranks still align consistently (low hours → low rank, low score → low rank) → **monotonic relationship** → **Spearman = 1.0**, same result as the sorted version.

Notice: As hours go up, rank goes up consistently → **monotonic relationship** → Spearman = 1.0

Key Advantages Over Pearson

Situation	Pearson	Spearman
Outliers present	✗ Distorted	✓ Robust
Non-linear but monotonic	✗ Underestimates	✓ Captures it
Ordinal data (rankings)	✗ Not appropriate	✓ Correct
Normal distribution	✓ More powerful	⚠ Slightly less powerful

LINEAR: $y = 2x + 3$ → Both Pearson AND Spearman ≈ 1.0

MONOTONIC: $y = x^3$ (curved) → Pearson < 1.0 , but Spearman ≈ 1.0

NON-MONOTONIC: $y = x^2$ → Both ≈ 0 (non-monotonic around zero)


```

x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
y_linear = 3 * x + 2 # perfect linear: y = 3x + 2
y_monotonic = x**3 # monotonic nonlinear: y = x^3
y_with_outlier = np.array([3, 6, 9, 12, 15, 18, 21, 24, 27, 200]) # outlier at end

datasets = [
    ('x', 'y_linear', y_linear, "Linear: y = 3x + 2"),
    ('x', 'y_monotonic', y_monotonic, "Monotonic: y = x^3"),
    ('x', 'y_with_outlier', y_with_outlier, "Linear + Outlier"),
]

print(f"\n{'Dataset':<30} {'Pearson r':>12} {'Spearman r':>12} {'Difference':>10}")
print("-" * 68)
for _, x, y, name in datasets:
    p = np.corrcoef(x, y)[0, 1]
    s = stats.spearmanr(x, y).statistic
    print(
        f"{'name':<30} {'p':>+12.4f} {'s':>+12.4f} {'← LARGE DIFF!' if abs(p-s) > 0.05 else '≈ same':>10}"
    )

```

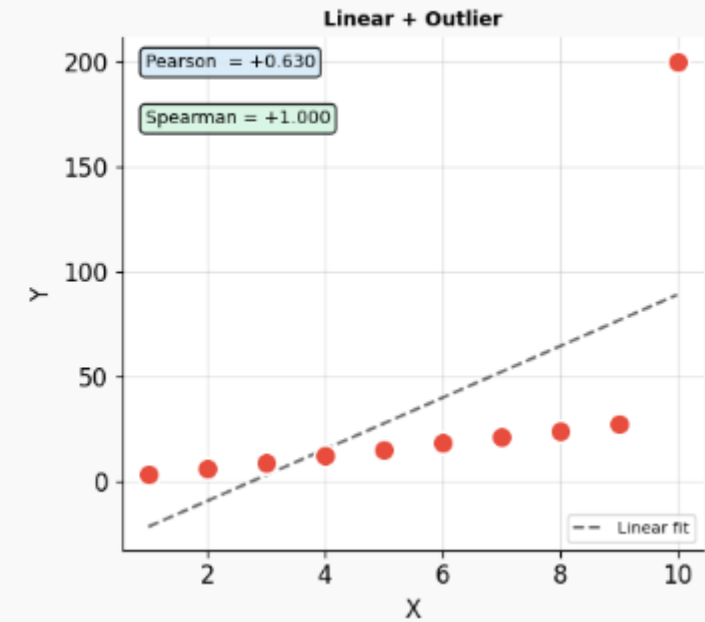
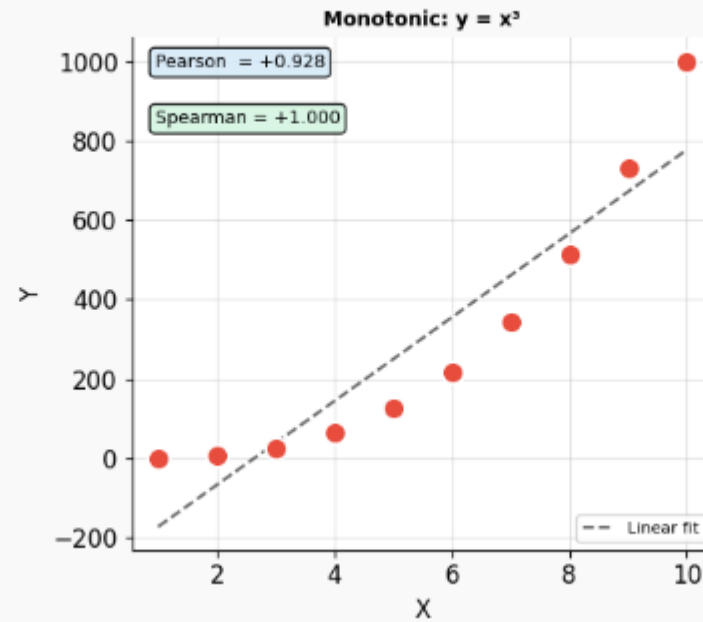
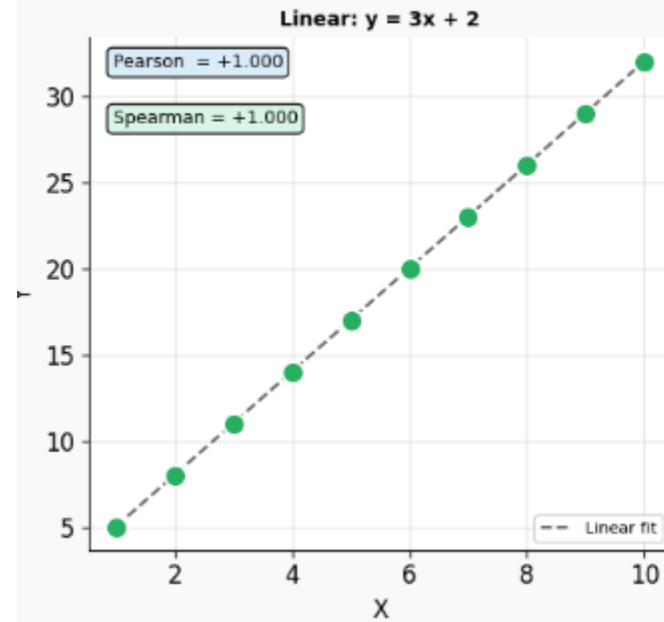
=====

EXAMPLE 1 – Pearson vs Spearman on Linear vs Monotonic Data

=====

Dataset	Pearson r	Spearman r	Difference
-----	-----	-----	-----
Linear: y = 3x + 2	+1.0000	+1.0000	≈ same
Monotonic: y = x ³	+0.9284	+1.0000	← LARGE DIFF!
Linear + Outlier	+0.6297	+1.0000	← LARGE DIFF!

Pearson vs Spearman — When They Disagree



💡 KEY INSIGHTS:

- Linear: Pearson $\approx$ Spearman ≈ 1.0 (both perfect)
- Monotonic: Spearman = 1.0 but Pearson < 1.0 (misses the nonlinear curve)
- Outlier: Pearson is pulled, Spearman stays robust

```
pearson_m = multi_df.corr(method='pearson').round(3)
spearman_m = multi_df.corr(method='spearman').round(3)
```

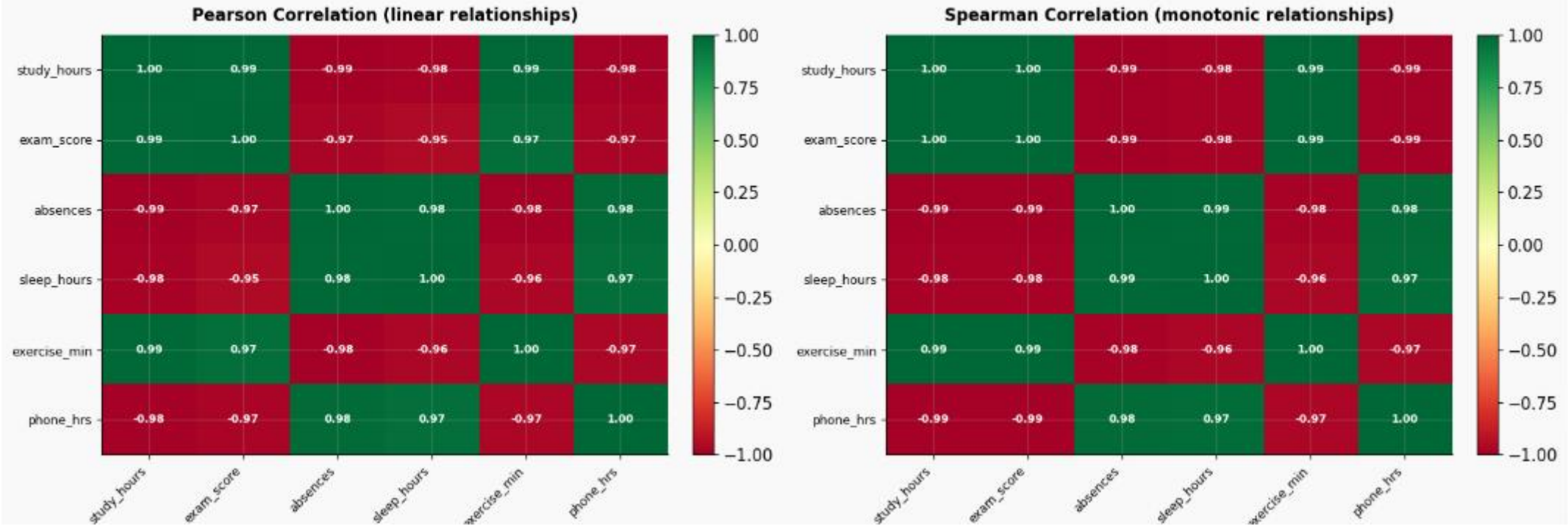
 Spearman Correlation Matrix:

	study_hours	exam_score	absences	sleep_hours	exercise_min	\
study_hours	1.000	1.000	-0.995	-0.982	0.993	
exam_score	1.000	1.000	-0.995	-0.982	0.993	
absences	-0.995	-0.995	1.000	0.988	-0.984	
sleep_hours	-0.982	-0.982	0.988	1.000	-0.961	
exercise_min	0.993	0.993	-0.984	-0.961	1.000	
phone_hrs	-0.986	-0.986	0.984	0.975	-0.968	

 Highly Correlated Pairs ($|r| > 0.8$) – Multicollinearity Risk:

study_hours	↔ exam_score	: r = +0.988
study_hours	↔ absences	: r = -0.993
study_hours	↔ sleep_hours	: r = -0.977
study_hours	↔ exercise_min	: r = +0.991
study_hours	↔ phone_hrs	: r = -0.985
exam_score	↔ absences	: r = -0.974
exam_score	↔ sleep_hours	: r = -0.953
exam_score	↔ exercise_min	: r = +0.968
exam_score	↔ phone_hrs	: r = -0.970
absences	↔ sleep_hours	: r = +0.985
absences	↔ exercise_min	: r = -0.985
absences	↔ phone_hrs	: r = +0.982
sleep_hours	↔ exercise_min	: r = -0.965
sleep_hours	↔ phone_hrs	: r = +0.966
exercise_min	↔ phone_hrs	: r = -0.967

Correlation Heatmaps — Pearson vs Spearman



How to read a heatmap:

- Dark green (close to +1): Strong positive correlation
- Dark red (close to -1): Strong negative correlation
- Yellow (close to 0): No correlation



Summary — Dimensions 5 through 8

Dimension	Key Concept	Main Tools	Watch Out For
● Outliers	Z-Score, IQR, Isolation Forest	<code>zscore()</code> , <code>quantile()</code> , <code>IsolationForest</code>	Choosing the right method for your distribution
● Timeliness	Freshness, Order, Gaps, Frequency	<code>is_monotonic_increasing</code> , <code>diff()</code> , <code>date_range()</code>	Missing days vs. expected dates
● Distribution	Skewness, Kurtosis, KS Test	<code>skew()</code> , <code>kurtosis()</code> , <code>kstest()</code>	Excess vs Pearson kurtosis convention
● Relationships	Pearson (linear) vs Spearman (monotonic)	<code>df.corr(method=...)</code> , <code>spearmanr()</code>	Pearson fails with outliers & non-linearity

"Your model is only as good as your data." — Garbage in, garbage out.